LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Advanced Topics in Parallel Computing

Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

Condition Synchronization



Condition Synchronization

- Most synchronization scenarios fall into two classes
  - **Atomicity**: make sure that a set of events appear to be executed atomically (can be achieved with mutual exclusion)
  - **Condition synchronization**: make sure that some condition is satisfied before continuing
- Condition synchronization means “wait until a condition is fulfilled”
  - “condition” usually refers to some state of the system (i.e., memory)
  - Can be realized in two ways: **spinning/busy-waiting** or **scheduler-based**
- Examples:
  - Wait until a flag is set to *true* before continuing
  - Wait until all threads have arrived at a barrier before continuing

Flag-Based Synchronization

```
class Flag {
  bool f = false;

set():
  f.store(true, RW||); // release ordering

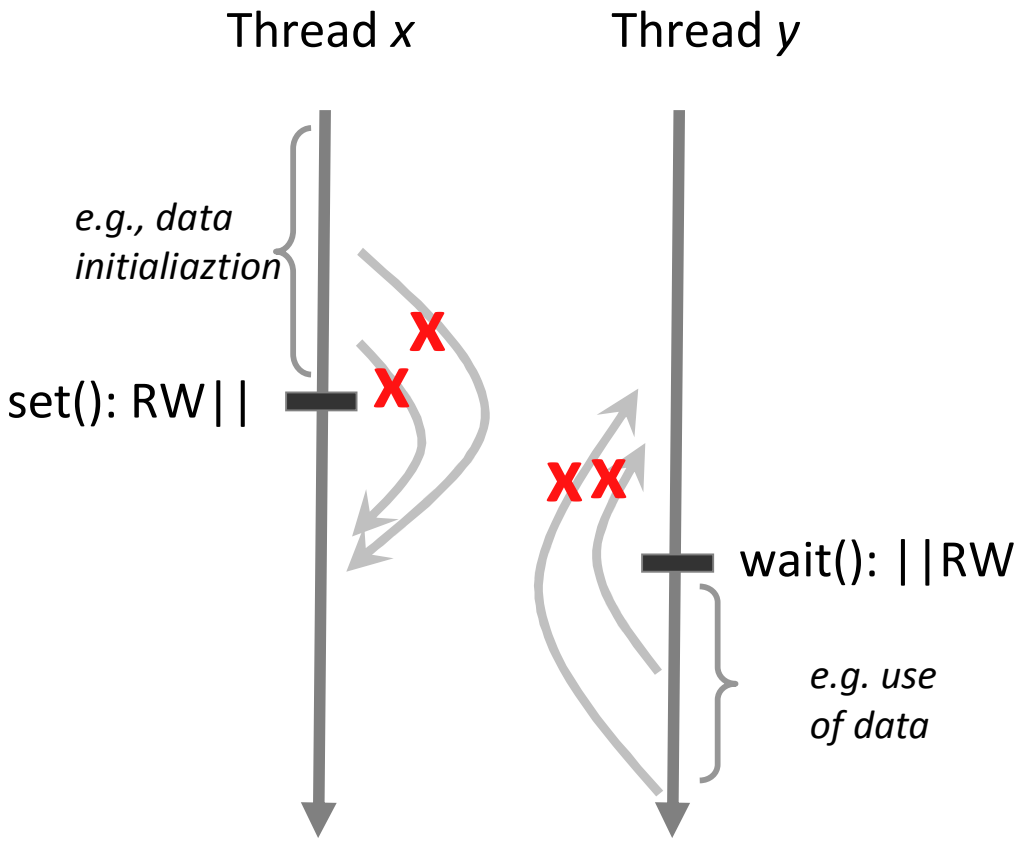
wait():
  while(!f.load()) {}  // spin
  fence(R||RW);        // acquire ordering
}
```

- **set()** and **wait()** are typically called in different threads (one that establishes a condition and another one that waits for it)

Acquire and Release Ordering for Flag

- **Release ordering** on set():
  - Required because a flag is typically used to indindicate that some operation (e.g. initialization of some data structure has finished)

- **Acquire ordering** on wait()
  - Required to ensure that subsequent operations do not begin before condition holds



Reset

```
class Flag {
  bool f = false;

set():
  f.store(true, RW||); // release ordering

wait():
  while(!f.load()) {}  // spin
  fence(R||RW);        // acquire ordering

reset():
  f.store(false, ||W);
}
```

- reset() functionality is needed if we want to use the flag more than once
  - ||W ordering is required because we need to make sure that a subsequent set() operation (which entails a write) is not reordered before the reset

Barrier Synchronization

- Barriers are found in most parallel programming approaches
  - C++ is late to the party (**std::barrier** only available since C++20)

- Semantics of a barrier:
  - **Wait** for all threads to arrive, then **continue**
  - Wait can be busy-waiting (spinning) or scheduler-based waiting

- Barriers are often used to separate computational **phases**
  - Initialization, main computation, output
  - Iterative phases of a numeric calculation

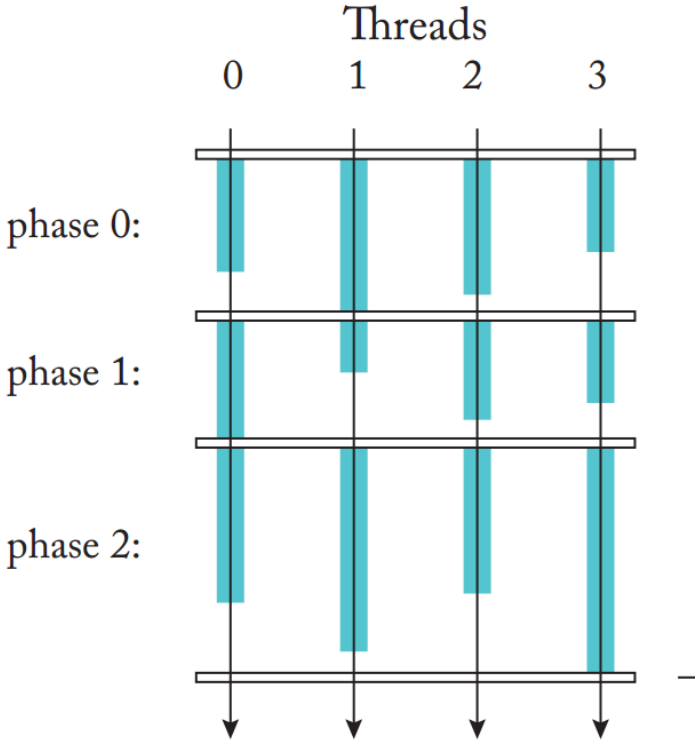
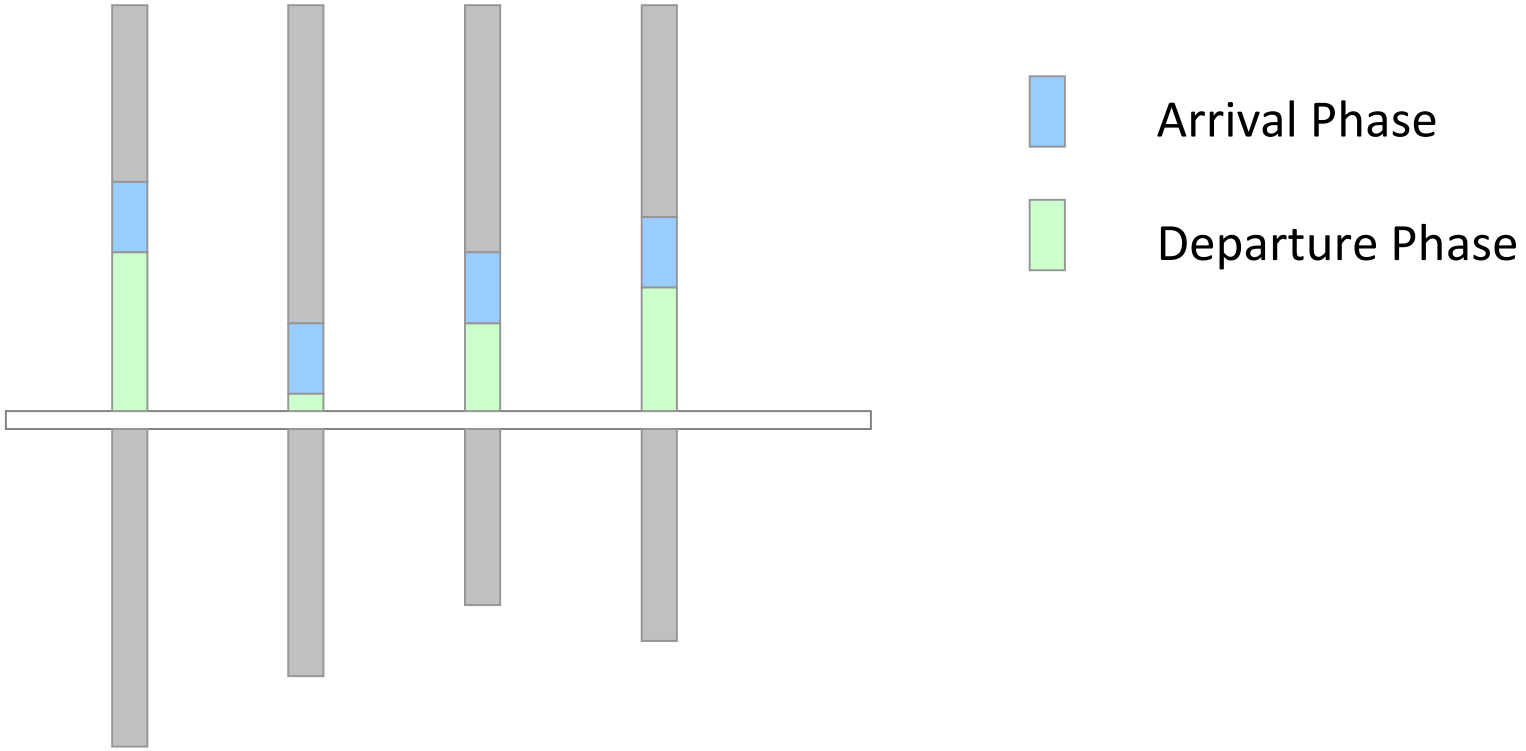


Image Source: Michael L. Scott "Shared-memory synchronization" Synthesis Lectures on Computer Architecture (2013)

Implementing Barriers (1)

- Implementation of a barrier
  - **Arrival phase**: Let everyone know you have arrived
  - **Waiting/departure phase**: Make sure everyone else has arrived, then depart



Implementing Barriers (2)

- Memory ordering and barriers
  - Typically we want to have **all changes made by a thread before a barrier be visible to all threads after the barrier**
  - Typically we don't want **operations after the barrier to take effect before the barrier, from the point of view of other threads**
  - Thus, a barrier typically acts as a **RW||RW fence**

  - Typically the **arrival** phase includes a (**RW|| reordering restriction**)
  - Typically the **departure** phase includes an (**||RW reordering restriction**)

Implementing Barrier Synchronization

- Centralized Barrier
  - Atomically increment/decrement a single counter
  - **Sense reversal** is used to make the barrier reusable
- Dissemination Barrier
  - Use read/write operations only and ideas from *information dissemination*
- Combining Tree Barrier
  - Avoid spinning on a single location by arranging (sub-)barriers in a tree
- Static Tree Barrier
  - Arrange threads in a tree (leaves and internal nodes)

Centralized Barrier (1)

- A deceptively simple idea:
  - Increment a counter **atomically** for each thread arriving at the barrier
  - Spin until the required number (total number of threads) is reached
  - Once the required number is reached, **reset the counter**

```
// Buggy Barrier
class Barrier {
  const int N = |T|; // total number of threads
  int count = 0;     // already arrived threads

arrive_and_wait():
  if FAI(&count, RW||) == N-1 {
    count.store(0); // prepare for next episode
  } else {
    while( count.load()!=N-1 ) {} // spin
  }
  fence(R||RW);
}
```

Problems with this Barrier

- **Q**: What's the problem with this approach?
- **A**: It's not guaranteed that the "triggering" value of **count** (N-1) will be seen by all threads before being reset to 0; others will spin forever
- **Solution**: Spin on a separate flag instead

Centralized Barrier (2)

- Second try
  - Once we reach the required number of threads, set a flag
  - This releases **all** waiting threads
  - But when should we reset the flag?

```
// Still a buggy barrier
class Barrier {
  const int N = |T|; // total number of threads
  int count = 0;     // already arrived threads
  bool wait = true;

arrive_and_wait():
  if FAI(&count, RW||) == N-1 {
    count.store(0);    // prepare for next episode
    wait.store(false); // release all threads
  } else {
    while( wait.load() ) {} // spin
  }
  fence(R||RW);
}
```

Problems with this Barrier

- **Q**: What's the problem with this approach?
- **A**: We can only reset the wait flag, once the last thread has been released. But one of the earlier released threads might already have called arrive_and_wait() again on the next barrier "episode" before that
- **Solution**: We need to make sure that consecutive barrier episodes don't interfere with each other
  - Some threads may just be leaving the barrier for the current phase
  - Other threads may already be arriving at the barrier for the next phase
- A simple counter, like in the last slide, is enough for a one-time barrier (aka a "latch", see later for more)

Sense-Reversing Barrier

- Idea:
  - Spin on boolean flag and invert its “sense” with each consecutive phase

```
class SenseReversingBarrier {
  const int N = |T|; // total number of threads
  int count = 0;     // already arrived

  bool sense = true;
  bool lsense[N] = {true,...}; // thread-local sense

arrive_and_wait():
  lsense[self] = !lsense[self]; // toggle local sense

  if FAI(&count, RW||) == N-1 {
    count.store(0); // prepare for next episode
    sense.store(lsense[self]); // release threads
  } else {
    while (sense.load() != lsense[self]) {} // spin
  }
  fence(R||RW);
}
```

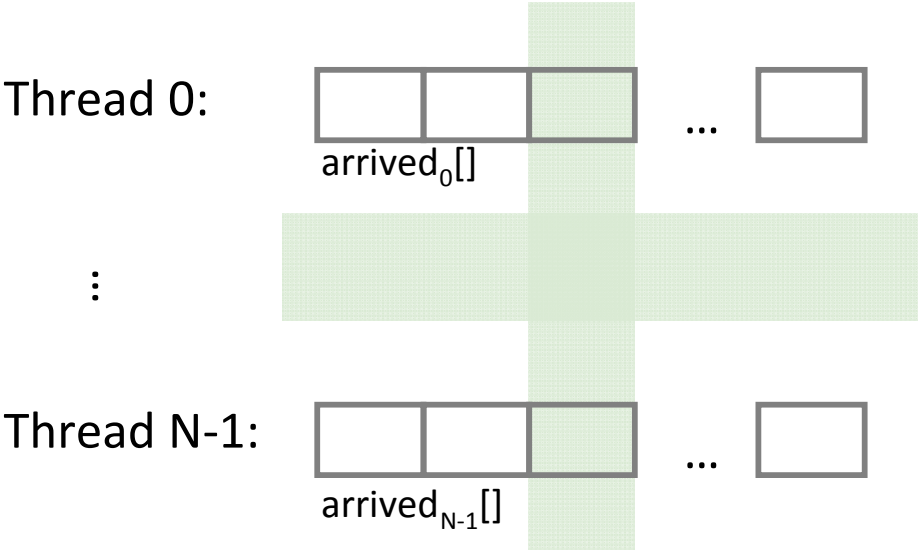
Sense-Reversing Barrier Discussion

- On a cache-coherent system the spinning will be cheap (**sense** will be cached locally)
  - Once a thread toggles **sense**, the local copies will be invalidated and the new value will be propagated to all waiting threads, releasing them
- The FAI can be a performance issue on a larger system (FAI requires writeable copy, so requires invalidating all other cached copies)
- **Solutions**:
  - Organize the barrier in the form of a tree – **combining tree barrier** and **static tree barrier**
  - Get rid of FAI entirely, use reads and writes only – **dissemination barrier**

Dissemination Barrier (1)

- Centralized barrier may suffer from performance degradation since threads execute FAI concurrently
  - Idea: Use load and store only and several rounds of **information dissemination**

- Dissemination barrier
  - Assume each thread **k** has an array **arrived$_k$**, where arrived$_k$[i] indicates whether thread i$\in$(0,…,N-1) has arrived at the barrier



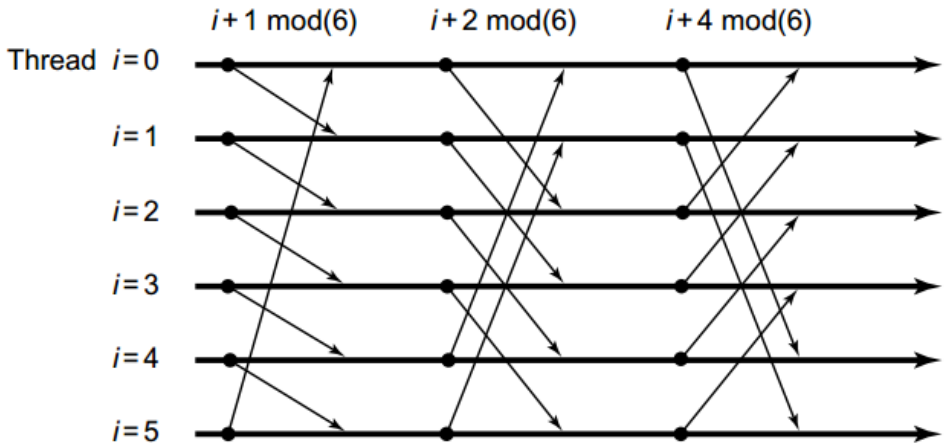
Thread k **arrives at barrier**:
set arrived$_j$[k]=true for all j$\in$(0,…,N-1)

Thread k **waits at barrier**:
for each j in (0,…N-1)
  while(arrived$_k$[j]!=true) {} //spin

Dissemination Barrier (2)

- Problems with the scheme on the previous slide
  - As stated, it works for one barrier episode only – so we need something similar to sense reversal again
  - The scheme is wasteful – not everyone needs to be informed by everyone directly

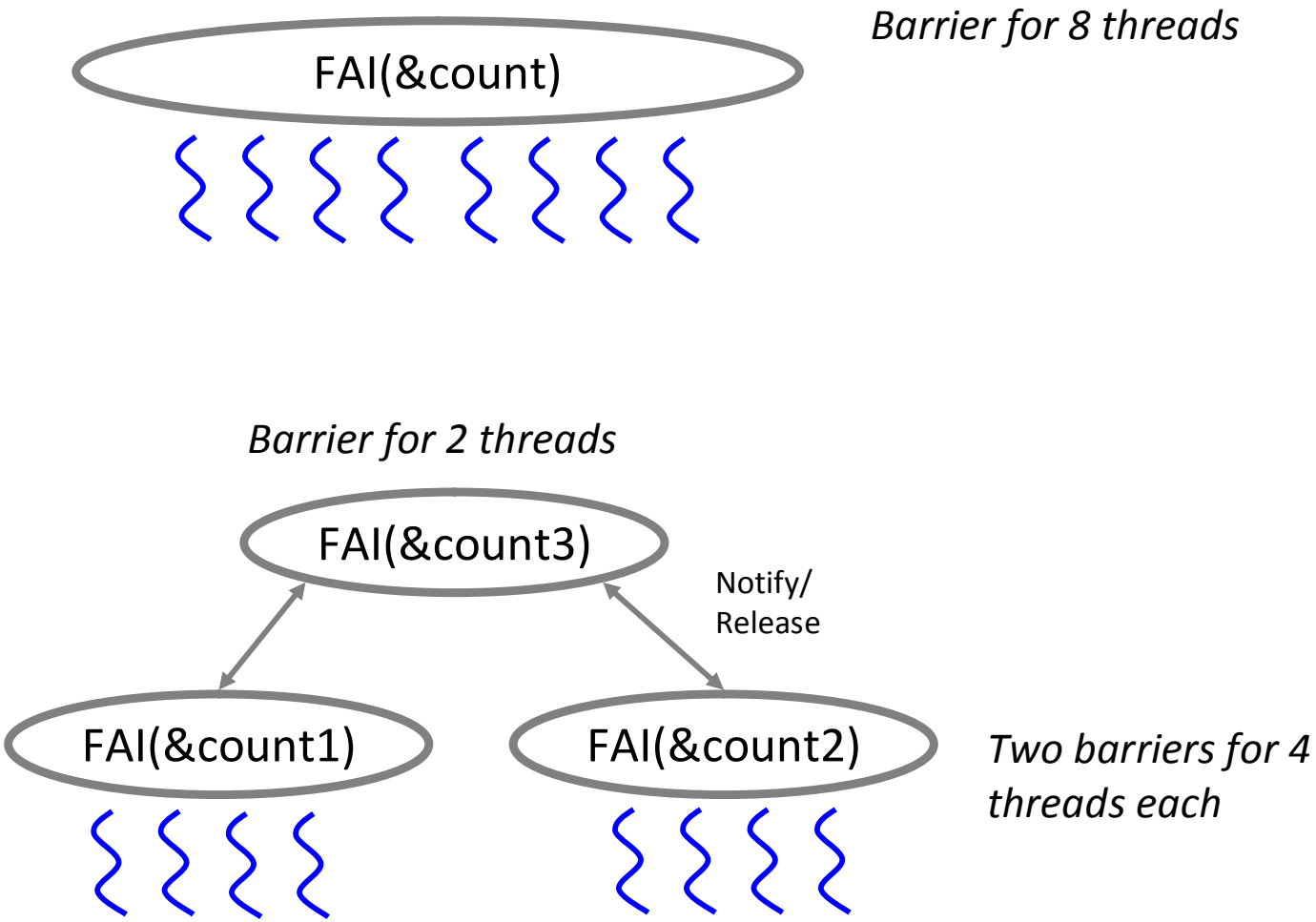
- We can do better:
  - ceil($\log_2$(N)) space per thread
  - ceil($\log_2$(N)) rounds of waiting



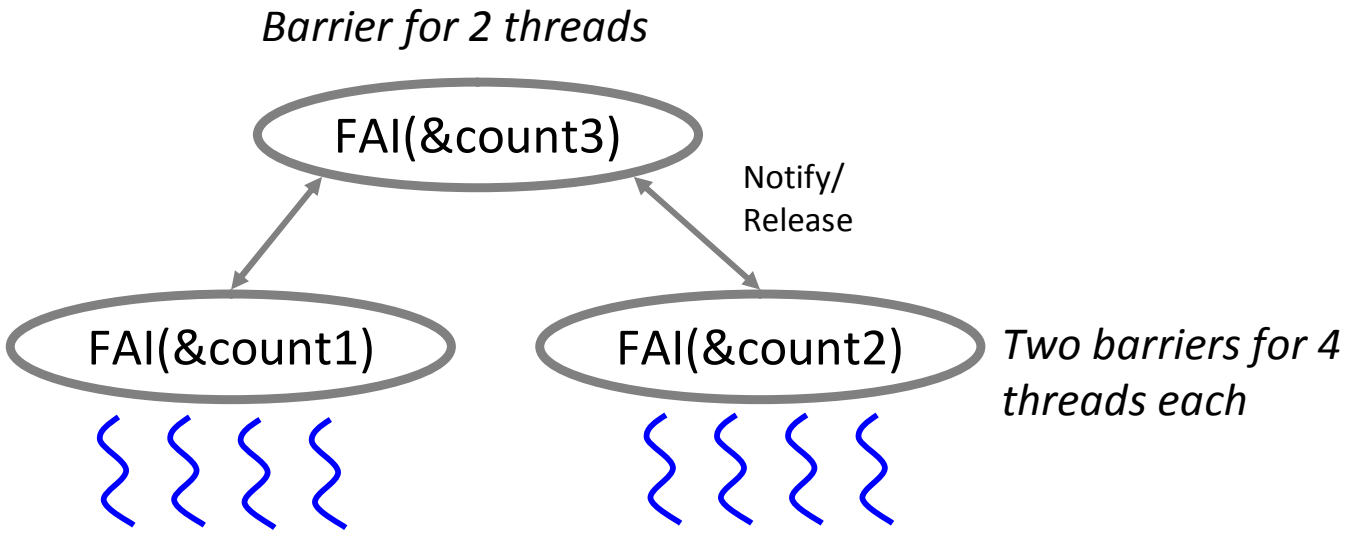
- ceil($\log_2$(N)) rounds: k=0,…,ceil($\log_2$(N)-1)
  - Thread i signals thread i+2^k
  - Works for any N (not only powers of 2)
  - After all rounds, every thread has heard from everyone else, directly or indirectly

Combining Tree Barrier (1)

- The centralized barrier may introduce overheads due to FAI on a single location
  - Idea: Split the barrier into sub-barriers and combine them in a tree



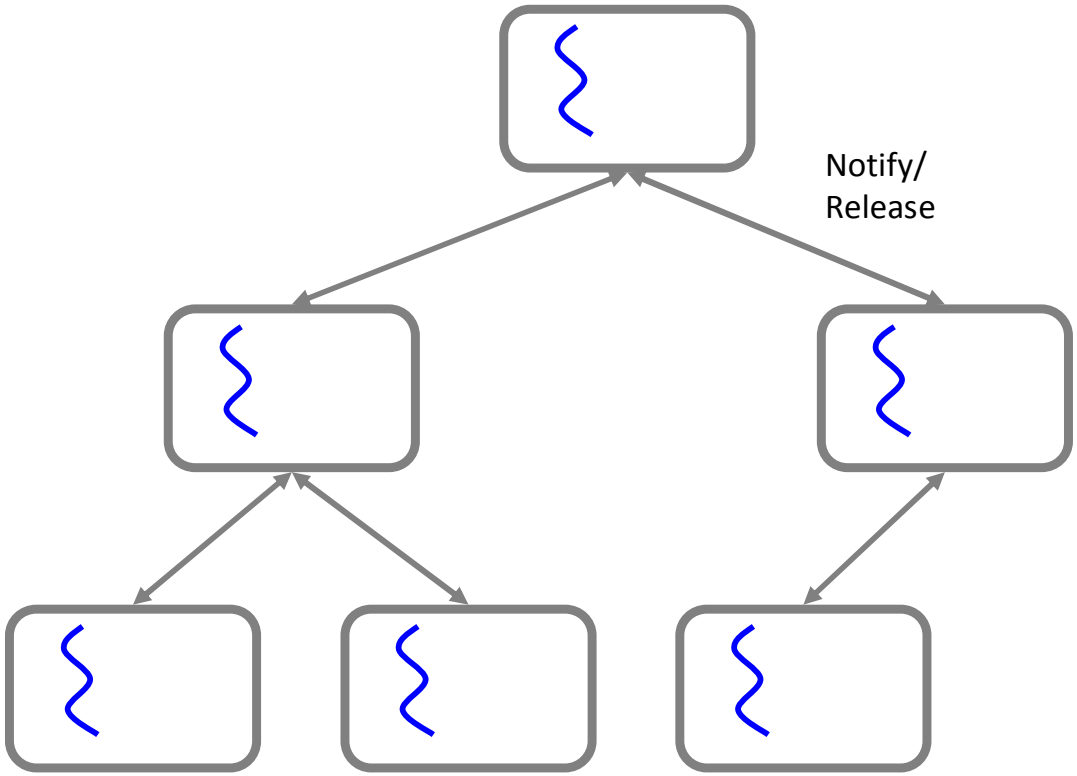
Combining Tree Barrier (2)



- On the way up the tree (towards the root)
  - One thread is chosen to notify the parent
  - For example, the last thread to arrive
- On the way down the tree (towards the leaves)
  - The parent node releases the child nodes
  - For example, by setting a global flag and simply relying on the cache coherence protocol

Static Tree Barrier

- Also uses a tree, but threads correspond to both **internal nodes and leaves**
  - Threads have a **predefined static location** in the tree, and thus a **predefined number of child nodes**
  - Arity can be different for the notify and the release tree



- Binary static tree barrier with 6 threads

Barrier Improvements and Extensions

- Split-phase barriers (aka. fuzzy barriers)
  - Separate the **arrival** and the **waiting part** of a barrier
  - Allows overlapping of “extra” work in phases with waiting

- Termination detection barrier
  - Useful for example in dynamic work scheduling situations (“terminate if all threads are done with their work”)

Split-Phase Barriers (1)

- General problem with iterative phases and barriers
  - A **single slow thread** forces all other threads to wait before the next phase can begin
  - So, we need to ensure good **load balancing**, which can be difficult

- Idea of **split-phase** or "**fuzzy**" barriers:
  - Perhaps not *all* work done by a thread in a phase is relevant to the other thread's execution in their next phase
  - Instead of the single **arrive_and_wait()** call, split the barrier functionality in two calls:
  - **arrive():** announce that you have arrived and done all work needed by others in the next phase
  - **wait():** wait for all other threads to call arrive()
  - The "extra" work (not needed by other threads) can be done between arrive() and wait()
  - Example for extra work: logging, computing statistics, etc.

Split-Phase Barriers (2)

```
Barrier barr;

while(...) {
  work();   // compute
  extra_work(); // logging
  barr.arrive_and_wait();
}
```

```
SplitPhaseBarrier barr;

while(...) {
  work();   // compute
  barr.arrive();
  extra_work(); // logging
  barr.wait();
}
```

Split-Phase Barrier Example

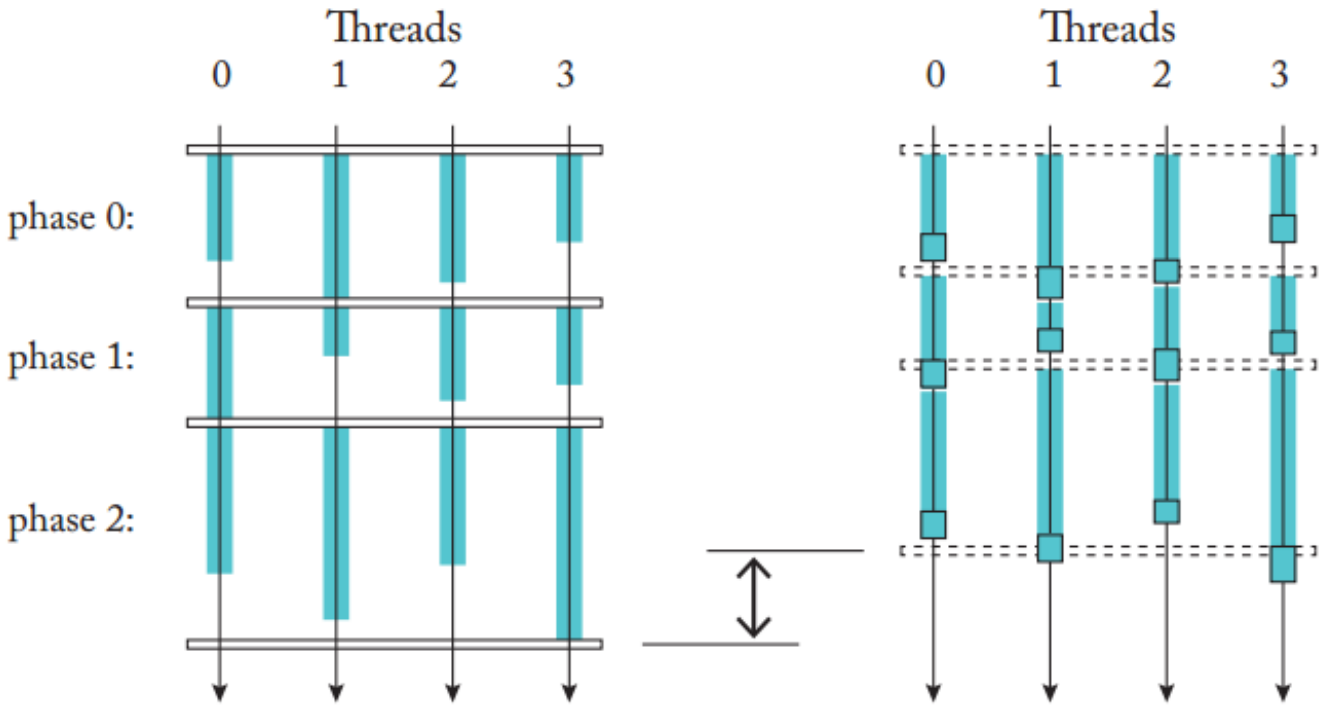
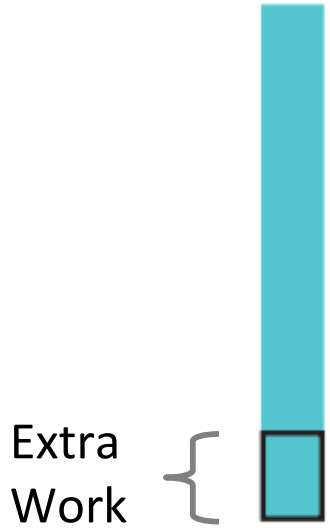


Image Source: Michael L. Scott "Shared-memory synchronization" Synthesis Lectures on Computer Architecture (2013)



Sense-Reversing Split-Phase Barrier

- *Some* barrier implementations can easily be turned into a split-phase version

```
class SenseReversingBarrier {
  const int N = |T|; // total number of threads
  int count = 0;     // already arrived

  bool sense = true;
  bool lsense[N] = {true,...}; // thread-local sense

arrive():
  lsense[self] = !lsense[self]; // toggle local sense

  if FAI(&count, RW||) == N-1 {
    count.store(0); // prepare for next episode
    sense.store(lsense[self]); // release threads
  }

wait():
  while (sense.load() != lsense[self]) {} // spin
  fence(R||RW);
}
```

Termination Detection Barrier

- Useful in situations where we have worker threads that may become active/inactive depending on work availability
  - Once **all** threads are inactive, **termination is detected**

```
class TerminationDetectionBarrier {
  int count = |T|;  // number of threads

set_active():
  FAI(&count, RW||);

set_inactive():
  FAD(&count, RW||);

wait():
  while(count.load()>0) {}  // spin
  fence(R||RW);

terminated():
  return count.load(||RW)==0;
}
```

Termination Detection and Workstealing

- Assume threads are executing tasks using workstealing load balancing
  - Work on local tasks, before stealing other tasks

```
TerminationDetectionBarrier td;
Task taskqueue[|T|];   // a task-queue per thread

void run() {
  while(task=taskqueue[self].head()) { // process local tasks
    task->run();
  }

  td.set_inactive();
  while(!td.terminated()) {
    if( taskqueue[victim].size()>0 )
      td.set_active()
    task = taskqueue[victim].tail(); // steal from random victim
    if(task)
      task->run();
    td.set_inactive();
  }
}
```

Barriers in C++

- C++ has barriers since C++20
  - Previous (experimental) versions were available in **std::experimental**
  - Slightly different functionality and syntax in final version

- Two forms are supported
  - **std::latch** Single-use barrier
  - **std::barrier** Reusable barrier

std::latch

- #include <latch>
  - An individual latch is a single-use object
- A latch **represents a counter counting down to zero**
  - Counter is **initialized in constructor**
  - There is **no reset** function and no way to increase the counter
  - **No way to query** the counter value…
  - Except to wait for it to reach 0 with **latch::wait()**
  - And to test if it has reached 0 with **latch::try_wait()**
- Decrementing the counter
  - **latch::count_down(n)** decrements the counter by n (default n=1)
  - **latch::arrive_and_wait(n)** counts down and waits
- A latch cannot be reused once the counter reaches 0

std::barrier (1)

- #include <barrier>
  - A reusable object who's lifetime consists of a **sequence of barrier phases**

- A barrier is represented by a counter ("expected count")
  - Initially set in constructor
  - Decremented by calls to arrive(), etc.
  - Once the count reaches 0, the **completion step** of the barrier is run (can be a user-supplied function)
  - After the completion step, the expected count is reset
  - Expected count for the next phase may be smaller for the next phase (a thread may decide not to participate in the next barrier episode)

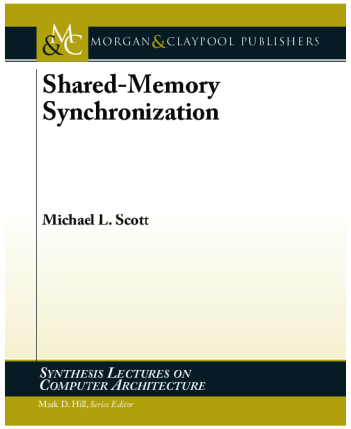
std::barrier (2)

- Functions
  - **barrier::arrive(n)** – decrement expected count by n (default 1)
  - **barrier::wait()** – wait for the arrival of the other threads
  - **barrier::arrive_and_wait()** – arrive() followed by wait()
  - **barrier::arrive_and_drop()** – arrive at the barrier and decrement expected count for next phase

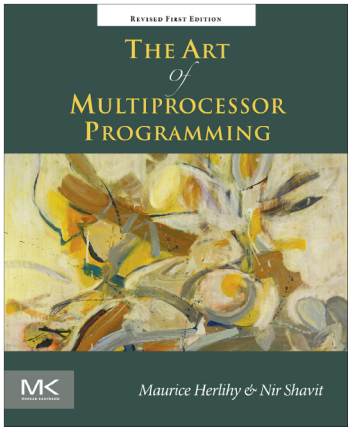
```
#define N 10 // number of threads

std::barrier barr(N);
#pragma omp parallel num_threads(N)
{
  for(int i=0; i<100; i++ ) {
    do_work();
    barr.arrive_and_wait();
  }
}
```

Further Reading



- Michael L. Scott *"Shared-memory synchronization"* Synthesis Lectures on Computer Architecture (2013)
  - Chapter 5 “Busy-wait Synchronization with Conditions”



- Maurice Herlihy, Nir Shavit *"The Art of Multiprocessor Programming"*
  - Chapter 17 “Barriers”